(Task 1 of 9) In this tutorial, we will review previously learned content.

0

Lispy |

(Task 2 of 9) What is the result of running this program?

Lispy [Run ▶]

```
(defvar a 5)
(deffun (k b)
  (set! b 3)
  a)
(k a)
```

Python

```
a = 5
def k(b):
    b = 3
    return a
print(k(a))
```

⚠ Scala 3 behaves differently.

`5`

2

You predicted the output correctly 🎉

3

The function call binds `b` to `5`. The variable assignment mutates the value of `b` to `3`, but `a` is still bound to `5`.

Click here to run this program in the Stacker.

Lispy |

(Task 3 of 9) What is the result of running this program?

Lispy [Run ▶]

```
(defvar ns (mvec 74 85))
(vec-set! ns 0 ns)
(vec-ref ns 1)
```

Scala 3

```
val ns = Buffer[Any](74, 85)
ns(0) = ns
println(ns(1))
```

`85`

5

You predicted the output correctly 🎉

6

`ns` is bound to a vector. `(vec-set! ns 0 ns)` replaces the `0`-th element of the vector with the vector itself. This is fine because a vector element can be any value, including itself. Besides, the vector is not copied, so the mutation finishes immediately.

Click here to run this program in the Stacker.

Lispy |

(Task 4 of 9) What is the result of running this program?

Lispy [Run ▶]

Pseudo

```
(defvar zz (mvec 88 88))
(deffun (f aa)
  (vec-set! aa 0 97)
  (vec-ref aa 0))
(defvar oo (f zz))
zz
```

```
let zz = vec[88, 88]
fun f(aa):
  aa[0] = 97
  return aa[0]
end
let oo = f(zz)
print(zz)
```

`#(97 88)`

8

You predicted the output correctly 🎉

9

The program first creates a two-element vector. The elements are both `88`. After that, the program defines a function `f`. The function call `(f zz)` replaces the first vector element with `97`. After that, the vector is printed. The vector now refers `97` and `88`, so the result is `#(97 88)`.

Click here to run this program in the Stacker.

Lispy |

(Task 5 of 9) What is the result of running this program?

Lispy [Run ▶]

```
(defvar n 7)
(deffun (foo)
  n)
(deffun (bar)
  (set! n 3)
  (foo))
(bar)
(set! n 5)
(foo)
```

Python

```
n = 7
def foo():
    return n
def bar():
    global n
    n = 3
    return foo()
print(bar())
n = 5
print(foo())
```

`3 5`

11

You predicted the output correctly 🎉

12

There is exactly one variable `n`. The `n` in `(set! n 3)` refers to that variable. Calling `bar` evaluates `(set! n 3)`, which mutates `n`. When the value of `n` is eventually printed, we see the new value.

Click here to run this program in the Stacker.

Lispy |

(Task 6 of 9) What is the result of running this program?

(Task 6 of 9) What is the result of running this program?

Lispy [Run ▶]

```
(defvar m (mvec 66))
(defvar z (mvec m 66 66))
(set! m (mvec 43))
z
```

Python

```
m = [ 66 ]
z = [ m, 66, 66 ]
m = [ 43 ]
print(z)
```

`#(#(43) 66 66)`

14

The answer is `#(#(66) 66 66)`.

▼ Textual explanation

You might think the `0`-th and first element of `z` refers to `m`. However, vectors refer to values, so it actually refers to the one-element vector, i.e., *the value of* `m`. In SMoL, variable assignments change *only* the mutated variables.

Click here to run this program in the Stacker.

15

Lispy |

What is the result of running this program?

Lispy [Run ▶]

```
(defvar x (mvec 0))
(defvar v (mvec 2 x 3))
(set! x (mvec 1))
v
```

Pseudo

```
let x = vec[0]
let v = vec[2, x, 3]
x = vec[1]
print(v)
```

`#(2 #(0) 3)`

17

You predicted the output correctly 🎉

18

Lispy |

(Task 7 of 9) What is the result of running this program?

Lispy [Run ▶]

```
(defvar p (mvec 72))
(defvar q p)
(vec-set! p 0 94)
q
```

Scala 3

```
val p = Buffer(72)
val q = p
p(0) = 94
println(q)
```

`#(72)`

20

The answer is `#(94)`.

21

▼ Textual explanation

You might think `p` and `q` refer to different vectors, so changing `p` doesn't change `q`. However, they refer to the same vector. In SMoL, a vector can be referred to by more

than one variable, and binding a vector to a new variable does not create a copy of that vector.

Click here to run this program in the Stacker.

Lispy |

What is the result of running this program?

Lispy [Run ▶]

```
(defvar a (mvec 37 26))
(defvar b a)
(vec-set! a 0 87)
b
```

Pseudo

```
let a = vec[37, 26]
let b = a
a[0] = 87
print(b)
```

23

```
#(87 26)
```

24

You predicted the output correctly 🎉

Lispy |

(Task 8 of 9) What is the result of running this program?

Lispy [Run ▶]

```
(defvar a 6)
(defvar b a)
(set! a 5)
a
b
```

Python

```
a = 6
b = a
a = 5
print(a)
print(b)
```

26

```
5 6
```

27

You predicted the output correctly 🎉

The first definition binds `a` to `6`. The second definition binds `b` to the value of `a`, which is `6`. The variable assignment mutates the binding of `a`, so `a` is now bound to `5`. But `b` is still bound to `6`.

Click here to run this program in the Stacker.

Lispy |

(Task 9 of 9) What is the result of running this program?

Lispy [Run ▶]

```
(defvar nn (mvec 66))
(defvar mm (mvec 77 nn))
(vec-set! nn 0 77)
mm
```

JavaScript

```
let nn = [ 66 ];
let mm = [ 77, nn ];
nn[0] = 77;
console.log(mm);
```

```
#(77 #(66))
```

29

The answer is `#(77 #(77))`.

▼ Textual explanation

You might think `nn` and the 1-th element of `mm` refer to different vectors, so changing `nn` doesn't change the 1-th element of `mm`. However, they refer to the same vector. In SMoL, a vector can be referred to by multiple places, and creating new vectors that refer to existing ones does not create new copies of the existing vectors.

Click here to run this program in the Stacker.

30

What is the result of running this program?

Lispy [Run ▶]

```
(defvar m (mvec 0))
(defvar v (mvec 52 m 53))
(vec-set! m 0 71)
v
```

JavaScript

```
let m = [ 0 ];
let v = [ 52, m, 53 ];
m[0] = 71;
console.log(v);
```

Lispy |

31

```
#(52 #(71) 53)
```

32

You predicted the output correctly 🎉

33

You have finished this tutorial 🎉

Please print the finished tutorial to a PDF file so you can review the content in the future. **Your instructor (if any) might require you to submit the PDF.**

Start time: 1781413563906